

CIFAKE: IMAGE CLASSIFICATION AND EXPLAINABLE IDENTIFICATION OF AI-GENERATED SYNTHETIC IMAGES

Submitted By: Sarthak Kalyani

B.Tech – Computer Science and Engineering

Email: cse23097@glbitm.ac.in

CHAPTER 1 — INTRODUCTION

1.1 Project Background

The rapid advancement of generative artificial intelligence has made it possible to create highly realistic synthetic images. These AI-generated images can closely resemble authentic photographs, making manual identification increasingly difficult. The CIFAKE project focuses on using deep learning-based image classification to distinguish between real and AI-generated synthetic images.

The project uses a Convolutional Neural Network (CNN) to automatically analyze an input image and classify it into one of two categories: REAL or FAKE. In addition to classification, the system incorporates Grad-CAM (Gradient-weighted Class Activation Mapping) to provide a visual explanation of the regions that influence the CNN's prediction.

The implemented system provides a web-based interface using Flask, allowing users to upload an image and receive a classification result along with an explainable visualization.

1.2 Problem Statement

The increasing availability of generative AI tools has made the creation of realistic synthetic images easier. Such images can potentially be used to spread misinformation or make it difficult to determine whether visual content is authentic.

Traditional manual inspection is unreliable because synthetic images can contain subtle characteristics that may not be easily noticed by humans. Therefore, there is a need for an automated image classification system that can identify AI-generated images and provide an understandable explanation for its prediction.

The proposed CIFAKE system addresses this problem by using a CNN-based classifier to distinguish real images from AI-generated synthetic images and by using Grad-CAM to visualize important image regions associated with the prediction.

1.3 Objectives

The main objectives of the project are:

1. To develop a deep learning-based system for distinguishing between real and AI-generated images.
2. To implement a CNN-based image classification model using the CIFAKE dataset.
3. To preprocess input images by resizing them to 32×32 pixels and normalizing pixel values.
4. To classify uploaded images into REAL or FAKE categories.
5. To integrate Grad-CAM for providing visual explanations of CNN predictions.

6. To develop a Flask-based web interface for image upload and prediction.
7. To evaluate the model using accuracy, precision, recall, F1-score and confusion matrix.

1.4 Scope of the Project

The scope of the project includes the development of a prototype system capable of analyzing image inputs and identifying whether they belong to the real or AI-generated category.

The system covers:

- Image uploading through a web interface.
- Image preprocessing using OpenCV.
- CNN-based classification.
- REAL/FAKE prediction.
- Grad-CAM-based visual explanation.
- Performance evaluation using standard classification metrics.
- Visualization of training and testing results.

The current implementation focuses specifically on image-level classification and does not attempt to identify the particular generative AI model or tool used to create a synthetic image.

1.5 Project Outcomes

The expected outcome is a functional AI-based image classification system that can automatically distinguish between real and synthetic images.

The implemented model was evaluated on a 20,000-image test set and achieved:

Metric	Result
--------	--------

Accuracy	95.93%
----------	--------

Precision	94.70%
-----------	--------

Recall	97.29%
--------	--------

F1-Score	95.98%
----------	--------

The system also provides Grad-CAM visualizations, allowing users to understand which regions of an image contributed to the model's classification.

Overall, the project demonstrates the application of CNN-based deep learning, computer vision, explainable AI and Flask web development to the problem of detecting AI-generated synthetic images.

CHAPTER 2 — SYSTEM OVERVIEW

2.1 Overview of the System

The CIFAKE system is a deep learning-based image classification application developed to identify whether an uploaded image is REAL or FAKE. The system combines a Convolutional Neural Network (CNN) with Explainable Artificial Intelligence (XAI) techniques to provide both a classification result and a visual explanation.

The trained model processes an input image after performing the required preprocessing operations. The image is resized to 32×32 pixels, converted into a numerical array, and normalized so that the pixel values lie between 0 and 1. The processed image is then passed to the CNN model for classification.

The system produces one of two outputs:

- REAL — the image is classified as an authentic image.
- FAKE — the image is classified as an AI-generated synthetic image.

In addition to the classification result, the system uses Grad-CAM (Gradient-weighted Class Activation Mapping) to generate a heatmap. This heatmap provides a visual indication of the image regions that contributed to the CNN's prediction.

The complete system is implemented as a Flask-based web application, allowing users to upload images through a browser and view the prediction and Grad-CAM visualization.

2.2 System Objectives

The system is designed to achieve the following objectives:

1. Automatically classify images into REAL and FAKE categories.
2. Use a CNN model to learn visual patterns associated with real and synthetic images.
3. Preprocess images consistently before classification.
4. Provide an explainable visualization using Grad-CAM.
5. Provide an easy-to-use web interface for image upload and prediction.
6. Evaluate the performance of the trained model using standard classification metrics.

2.3 Working Principle

The overall working of the CIFAKE system can be summarized as follows:

Step 1 — Image Upload:

The user selects and uploads an image through the Flask web interface.

Step 2 — Image Validation:

The uploaded file is checked to ensure that it can be read as a valid image.

Step 3 — Preprocessing:

The image is resized to 32×32 pixels and its pixel values are normalized by dividing them by 255.

Step 4 — CNN Classification:

The processed image is passed to the trained CNN model. The model produces class probabilities for the two classes.

Step 5 — Prediction:

The class with the highest probability is selected and displayed as either REAL or FAKE.

Step 6 — Grad-CAM Explanation:

Grad-CAM is applied to the trained model to generate a heatmap showing the regions associated with the prediction.

Step 7 — Result Display:

The Flask application displays the predicted class together with the Grad-CAM visualization.

2.4 System Inputs and Outputs

Component	Description
Input	User-uploaded image
Image Format	Supported readable image formats
Image Size	Resized to 32×32 pixels
Preprocessing	Pixel normalization between 0 and 1
Classification	REAL or FAKE
Explanation	Grad-CAM heatmap
Output	Prediction and explainable visualization

2.5 Dataset Overview

The processed dataset contains an equal number of samples in both classes: REAL: 50,000, FAKE: 50,000

The processed NumPy dataset was verified with the following dimensions:

- Images: 100,000
- Image dimensions: $32 \times 32 \times 3$
- Classes: 2
- Training images: 80,000
- Testing images: 20,000

The processed dataset contains an equal number of samples in both classes:

- REAL: 50,000
- FAKE: 50,000

This provides a balanced dataset for training and evaluation.

2.6 System Performance

The implemented model was evaluated using a test set containing 20,000 images. The obtained results were:

Metric	Result
Accuracy	95.93%
Precision	94.70%
Recall	97.29%
F1-Score	95.98%

These results demonstrate that the implemented CNN model is capable of distinguishing between real and synthetic images with high classification performance.

CHAPTER 3 — LITERATURE SURVEY

3.1 Introduction

The increasing use of generative artificial intelligence has created a need for reliable methods to identify AI-generated synthetic images. Researchers have investigated deep learning-based classification methods, convolutional neural networks and explainable AI techniques for detecting synthetic visual content.

The literature reviewed for this project provides the foundation for using CNN-based image classification together with Grad-CAM-based visual explanations.

3.2 CIFAKE Dataset and Image Classification

The CIFAKE research work focuses on distinguishing between real and synthetic images using deep learning-based image classification. The work demonstrates that CNN-based models can learn visual patterns that help distinguish AI-generated images from authentic images.

The dataset used in the research contains real and synthetic images and provides a suitable benchmark for binary image classification. The approach provides the basis for the REAL/FAKE classification implemented in this project.

3.3 Convolutional Neural Networks for Image Classification

Convolutional Neural Networks are widely used for image classification because they can automatically learn spatial features from images.

A CNN typically consists of convolutional layers, pooling layers and fully connected layers. Convolutional layers extract visual features, while pooling reduces the spatial dimensions of the extracted feature maps.

For the CIFAKE system, a Modified CNN2D architecture is used. The architecture contains two convolutional layers, two max-pooling layers, dropout layers and fully connected layers before the final two-class softmax output.

3.4 Explainable Artificial Intelligence

Although deep learning models can achieve high classification performance, their internal decision-making process can be difficult to interpret. This has resulted in increased interest in Explainable Artificial Intelligence (XAI).

XAI techniques attempt to provide information about why a machine learning model produced a particular prediction.

In this project, Grad-CAM is used as the explainability technique.

3.5 Grad-CAM

Grad-CAM, or Gradient-weighted Class Activation Mapping, is a visual explanation technique for CNN-based models.

It generates a class-specific activation map that highlights regions of an input image associated with the model's prediction. The resulting visualization can be represented as a heatmap and displayed alongside the original image.

Grad-CAM is particularly useful for image classification because it provides a visual interpretation of the features used by the CNN.

In the CIFAKE system, Grad-CAM is integrated into the prediction pipeline so that the user receives both the classification result and an associated visualization.

3.6 Comparison with the Proposed System

The existing research provides the theoretical and methodological foundation for the implemented CIFAKE system. However, this project focuses on creating an end-to-end usable application by integrating the trained CNN model into a Flask web interface.

Aspect	Existing Research	Proposed CIFAKE System
Image classification	CNN-based classification	CNN-based REAL/FAKE classification
Dataset	CIFAKE dataset	Processed CIFAKE dataset
Image preprocessing	Image resizing and normalization	32×32 resizing and normalization
Explainability	Grad-CAM	Grad-CAM integrated with prediction
User interface	Research implementation	Flask web application
Output	Classification result	Classification + Grad-CAM visualization
Evaluation	Classification metrics	Accuracy, precision, recall, F1-score and confusion matrix

3.7 Research Gap

Many image classification approaches focus primarily on achieving high classification accuracy. However, a classification result alone does not explain why the model reached a particular decision.

The implemented system addresses this limitation by combining classification with Grad-CAM visualization. This provides an additional interpretability layer and makes the system more suitable for demonstrating explainable AI concepts.

The project also integrates the trained model into a web-based Flask application, allowing users to interact with the classification system rather than using the model only through a development environment.

3.8 Summary

The literature survey establishes the relevance of CNN-based image classification for detecting synthetic images and demonstrates the importance of explainability in deep learning systems.

The reviewed concepts of CNN-based classification and Grad-CAM form the technical foundation of the CIFAKE project. These concepts are combined in the implemented system to provide REAL/FAKE classification together with an explainable visualization.

CHAPTER 4 — TECHNOLOGIES USED

4.1 Introduction

The CIFAKE system combines machine learning, computer vision and web development technologies to provide an end-to-end AI-generated image detection application.

The major technologies used in the project are Python, TensorFlow/Keras, OpenCV, NumPy, Scikit-learn, Matplotlib, Flask and SQLite.

4.2 Python

Python is the primary programming language used for implementing the project.

It is used for:

- Dataset processing
- Image preprocessing
- CNN model implementation
- Model evaluation
- Grad-CAM generation
- Flask web application development
- Result visualization

Python provides libraries and frameworks that simplify the development of machine learning and computer vision applications.

4.3 TensorFlow and Keras

TensorFlow and Keras are used for developing and executing the CNN-based image classification model.

Keras provides high-level APIs for defining the neural network architecture, training the model and loading the trained model weights.

The implemented model uses:

- Conv2D
- MaxPooling2D
- Dropout
- Flatten
- Dense
- Softmax

The trained model is stored as:

model/extension_weights.hdf5

4.4 OpenCV

OpenCV is used for image processing and computer vision operations.

In this project, OpenCV is used to:

- Read uploaded images.
- Resize images to 32×32 pixels.
- Convert image representations when required.
- Process images before CNN inference.
- Prepare images for visualization.

4.5 NumPy

NumPy is used for numerical operations and image-array manipulation.

The processed dataset is stored as NumPy arrays containing image pixels and their corresponding class labels.

The project uses:

model/X.txt.npy

model/Y.txt.npy

for storing the processed image data and labels.

4.6 Scikit-learn

Scikit-learn is used primarily for model evaluation and dataset splitting.

The project uses Scikit-learn functionality for:

- Train-test splitting
- Accuracy calculation
- Precision calculation
- Recall calculation

- F1-score calculation
- Confusion matrix generation

An 80:20 train-test split was used for the final evaluation.

4.7 Matplotlib

Matplotlib is used for generating graphical representations of the model's performance.

The project uses it to generate:

- Training and validation accuracy graphs.
- Training and validation loss graphs.
- Confusion matrix visualization.
- Grad-CAM-related visualizations.

These graphs are useful for understanding model performance and training behaviour.

4.8 Flask

Flask is used as the web application framework.

The Flask application provides routes for:

- Home page
- Login/signup pages
- Image upload
- Prediction
- Project information

The /predict route handles image classification requests. It receives the uploaded image, processes it, passes it through the trained CNN and displays the prediction and Grad-CAM visualization.

4.9 SQLite

SQLite is used for storing user registration information in the web application.

The database file used by the application is:

signup.db

The application stores basic user registration information such as username, email, mobile number and name. Authentication-related information is maintained through the SQLite database

4.10 HTML and Web Interface

HTML templates are used to create the user-facing web pages of the Flask application.

The project contains HTML templates for pages such as:

- Home
- Login
- Signup
- Validation
- Prediction results
- Project information

The web interface allows users to interact with the trained machine learning model without directly executing Python commands.

4.11 Development Environment

The project was developed and tested using:

Technology	Purpose
Python	Main programming language
TensorFlow/Keras	CNN model development
OpenCV	Image processing
NumPy	Numerical and image-array operations
Scikit-learn	Evaluation and dataset splitting
Matplotlib	Graphs and visualizations
Flask	Web application
SQLite	User information storage
Visual Studio Code	Development environment
Python Virtual Environment Dependency isolation	

4.12 Hardware and Software Requirements

Hardware Requirements

The project can be executed on a standard computer with:

- Multi-core processor
- Minimum 8 GB RAM recommended
- Sufficient storage for the dataset and model
- Internet connection for initial package installation

A dedicated GPU is not required for running the trained model. During testing, TensorFlow executed using CPU because CUDA/NVIDIA GPU libraries were not available on the development system.

Software Requirements

- Windows/Linux operating system
- Python
- TensorFlow
- Keras
- OpenCV
- NumPy
- Scikit-learn
- Matplotlib
- Flask
- SQLite
- Visual Studio Code or another Python-compatible IDE

CHAPTER 5 — ARCHITECTURE AND DESIGN

5.1 System Architecture

The CIFAKE system follows a pipeline-based architecture in which the user interacts with the application through a Flask web interface. The uploaded image is processed and passed to the trained CNN model for classification. After the prediction is generated, Grad-CAM is used to provide a visual explanation of the model's decision.

CIFAKE: Architecture and Workflow

Explainable Identification of AI-Generated Synthetic Images using Modified CNN2D and Grad-CAM

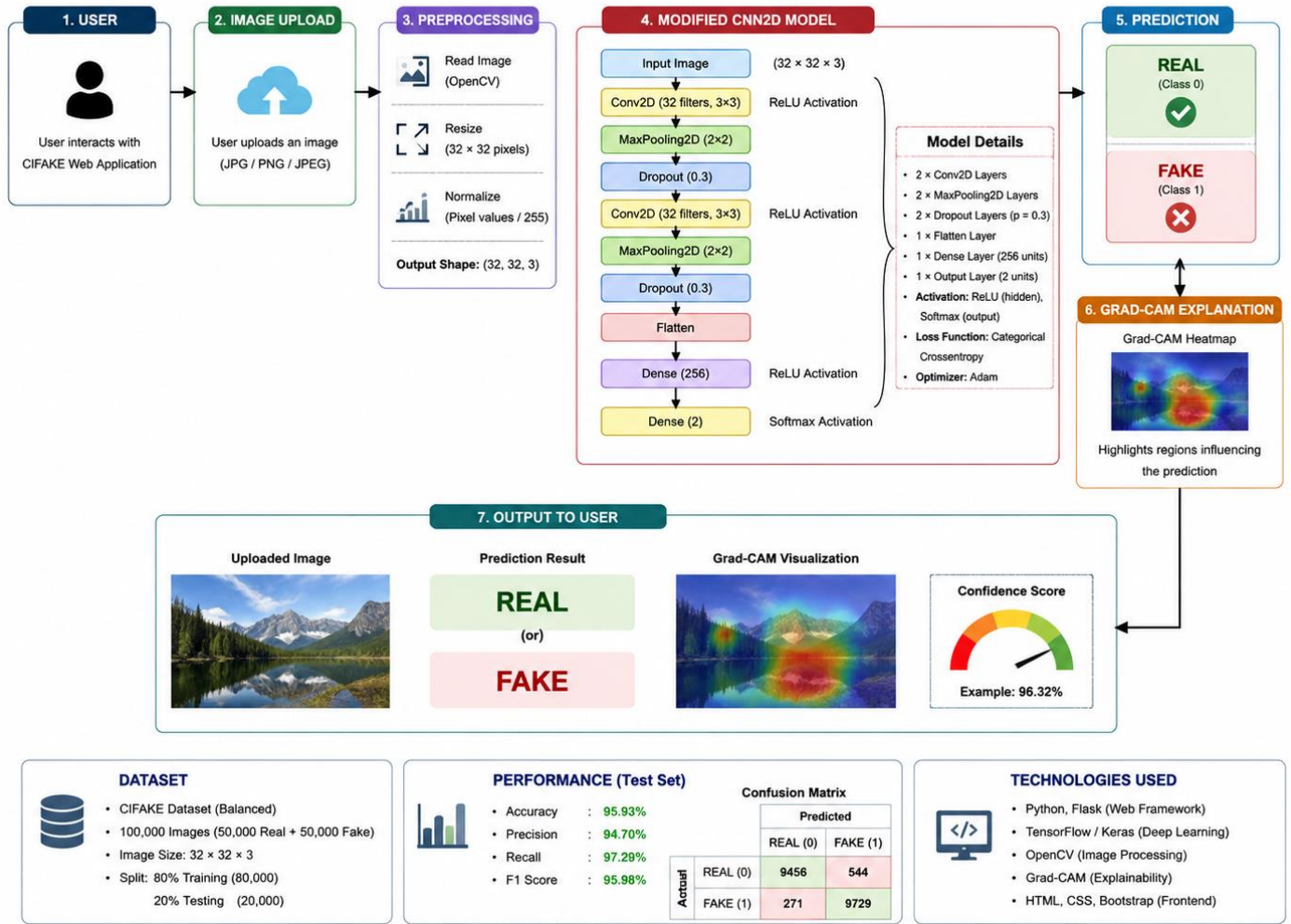


Figure 5.1: System Architecture of the Proposed CIFAKE System

The major stages of the system are:

User



Flask Web Interface



Image Upload



Image Preprocessing



Modified CNN2D Model



REAL / FAKE Prediction



Grad-CAM Explanation



Result Display

5.2 System Components

5.2.1 User Interface

The user interface provides the entry point to the application. Users can access the web application through a browser and upload an image for classification.

The interface also provides access to the project's information and authentication-related pages.

5.2.2 Flask Backend

Flask acts as the backend framework of the application. It manages HTTP requests, receives uploaded images and connects the web interface with the trained machine learning model.

The /predict route is responsible for handling image classification requests.

5.2.3 Image Preprocessing Module

The preprocessing module prepares uploaded images for the CNN.

The image processing pipeline includes:

1. Reading the uploaded image.
2. Validating and decoding the image.
3. Resizing it to 32×32 pixels.
4. Converting the image into a numerical array.
5. Converting pixel values to floating-point values.
6. Normalizing pixel values by dividing by 255.

This ensures that the input provided to the model follows the same basic format used during model training.

5.2.4 Modified CNN2D Model

The Modified CNN2D model performs the main classification task.

The architecture implemented in the project is:

Input Image

$32 \times 32 \times 3$



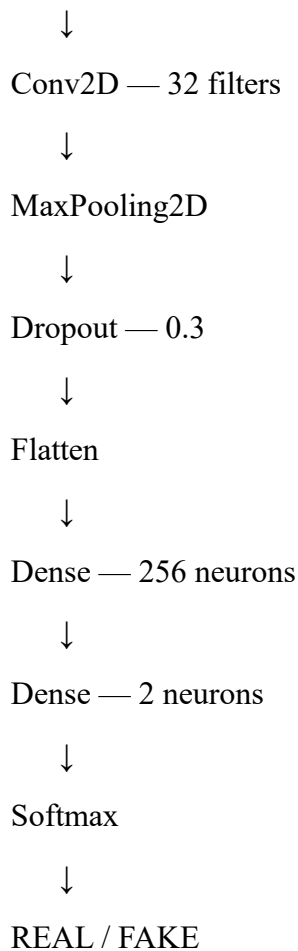
Conv2D — 32 filters



MaxPooling2D



Dropout — 0.3



The convolutional layers extract visual features from the input image. Pooling reduces the spatial dimensions of the feature maps, while dropout helps reduce the possibility of overfitting.

The flattened features are passed to a dense layer containing 256 neurons. The final two-neuron softmax layer produces the classification output.

5.2.5 Grad-CAM Module

The Grad-CAM module provides an explainability layer to the classification system.

After the CNN generates a prediction, activation information from the convolutional portion of the network is used to generate a heatmap.

The heatmap highlights regions associated with the model's learned activation and is displayed alongside the original image.

This allows the system to provide more information than a simple REAL or FAKE label.

5.2.6 Result Display Module

The result module combines the prediction and visualization generated by the system.

The output includes:

- Predicted class.
- Original uploaded image.
- Grad-CAM visualization.

For example:

Predicted As : REAL

or:

Predicted As : FAKE

5.3 Data Flow

The data flow through the system can be represented as:

Image



Upload



Validation



OpenCV Processing



$32 \times 32 \times 3$ Image



Normalization



CNN Model



Class Probability



REAL / FAKE



Grad-CAM



Visualization

The same preprocessing approach is used during inference so that the uploaded image is represented in a format compatible with the trained model.

5.4 Model Design Considerations

The architecture was designed to provide a relatively lightweight CNN suitable for the project's image classification task.

The use of two convolutional layers allows the model to learn visual patterns while keeping the architecture comparatively simple. Max-pooling reduces feature-map dimensions and computational requirements.

Dropout layers with a rate of 0.3 are included as a regularization mechanism. The final softmax layer enables binary classification between the REAL and FAKE categories.

5.5 Web Application Design

The Flask application contains multiple routes that provide different functions.

Route	Function
/	Main/index page
/home	Application dashboard
/logon	Signup page
/login	Login page
/predict	Image classification
/about	Project information
/notebook	Notebook page

The /predict route is the primary machine-learning route. It receives the uploaded image, performs preprocessing, loads the trained model weights, performs classification, generates the Grad-CAM visualization and returns the result page.

5.6 Database Design

SQLite is used for storing user registration information.

The application uses the signup.db database and an info table for storing registered-user information.

The stored fields include:

- Username
- Email
- Password
- Mobile number
- Name

The database provides basic persistence for the authentication component of the web application.

5.7 Design Summary

The proposed architecture integrates machine learning and web technologies into a single application.

The CNN performs the image classification, while Grad-CAM provides explainability. Flask connects these machine-learning components to a browser-based interface, allowing users to upload images and receive the prediction and explanation through the same application.

CHAPTER 6 — IMPLEMENTATION

6.1 Introduction

The CIFAKE system was implemented using Python, TensorFlow/Keras, OpenCV, Scikit-learn, Matplotlib and Flask. The implementation combines image preprocessing, CNN-based classification, Grad-CAM explainability and a web-based user interface.

The trained CNN model is stored as model weights and loaded by the Flask application during prediction. This avoids retraining the model whenever the web application is started.

6.2 Dataset Loading

The dataset images are loaded from the Dataset directory, which contains two classes:

Dataset/

├── FAKE/

└── REAL/

Each image is read using OpenCV and resized to 32×32 pixels.

The processed images are stored in a NumPy array X, while their corresponding class labels are stored in Y.

The processed arrays are saved for reuse:

model/X.txt.npy

model/Y.txt.npy

This reduces the need to repeatedly process all images when running subsequent experiments.

6.3 Image Preprocessing

The image data is converted into floating-point representation before normalization.

The normalization operation is:

$$X = X / 255$$

This converts the pixel values from the original 0–255 range to approximately 0–1.

The dataset is then shuffled before the train-test split is performed.

An 80:20 split is used:

80% → Training

20% → Testing

For the final evaluation, this corresponds to:

Training images : 80,000

Testing images : 20,000

6.4 CNN Model Implementation

The Modified CNN2D model is implemented using Keras.

The architecture consists of:

1. Conv2D layer with 32 filters.
2. MaxPooling2D layer.
3. Dropout layer with a rate of 0.3.
4. Second Conv2D layer with 32 filters.
5. Second MaxPooling2D layer.
6. Second Dropout layer with a rate of 0.3.
7. Flatten layer.
8. Dense layer containing 256 neurons.
9. Two-neuron softmax output layer.

The model is compiled using the Adam optimizer and categorical cross-entropy loss.

6.5 Model Training

The model is trained using the processed training dataset.

A batch size of 32 and a maximum of 15 epochs are used in the training implementation.

Validation data is used during training to monitor the model's performance.

The trained weights are saved using a model checkpoint:

model/extension_weights.hdf5

The training history is also saved:

model/extension_history.pkl

The saved history contains:

- Training loss
- Validation loss
- Training accuracy
- Validation accuracy

This allows the training performance to be analyzed without retraining the model.

6.6 Model Evaluation

After training, the model is evaluated on the test dataset.

Predictions are generated using the trained model and converted into class labels using the class with the highest predicted probability.

The following evaluation metrics are calculated:

- Accuracy
- Precision

- Recall
- F1-score
- Confusion matrix

The final evaluation on the 20,000-image test set produced:

Metric Score

Accuracy 95.93%

Precision 94.70%

Recall 97.29%

F1-Score 95.98%

6.7 Confusion Matrix Generation

A separate evaluation script was used to generate the confusion matrix.

The final confusion matrix was:

[[9456 544]

[271 9729]]

The matrix represents the number of correctly and incorrectly classified samples for the two classes.

The confusion matrix was saved as:

results/confusion_matrix.png

This visualization is used in the Results and Discussion chapter.

6.8 Training Graph Generation

The saved training history was used to generate graphical representations of model training.

Two graphs were generated:

results/training_validation_accuracy.png

results/training_validation_loss.png

The accuracy graph shows the change in training and validation accuracy across epochs.

The loss graph shows the change in training and validation loss across epochs.

The highest validation accuracy recorded in the saved training history was 93.89% at epoch 9.

At that epoch:

- Training accuracy: 93.10%
- Validation accuracy: 93.89%
- Training loss: 0.1726
- Validation loss: 0.1615

6.9 Flask Application Implementation

The trained model is integrated into a Flask web application.

When the application starts, the CNN architecture is reconstructed and the saved weights are loaded from:

model/extension_weights.hdf5

The user can then access the application through a web browser.

The main prediction functionality is implemented through the:

/predict

route.

6.10 Image Upload and Prediction

When a user uploads an image, the Flask application receives the uploaded file.

The image is stored temporarily and processed using OpenCV.

The image is resized to:

$32 \times 32 \times 3$

and normalized before being passed to the trained CNN.

The model generates the class probabilities and the class with the highest probability is selected.

The result is displayed as either:

Predicted As : REAL

or:

Predicted As : FAKE

6.11 Grad-CAM Implementation

After the classification step, the system generates a Grad-CAM-related visualization.

The implementation creates an auxiliary Keras model based on the trained CNN and extracts activation information from the convolutional portion of the network.

The resulting activation map is displayed as a heatmap alongside the original image.

This provides an additional visual explanation of the model's learned response to the input image.

6.12 Error Handling

Image validation and error handling were incorporated into the application to prevent invalid files from causing an application crash.

During development, an issue was encountered where OpenCV returned an empty image for an uploaded file. This caused the cv2.resize() operation to fail.

The upload and image-processing implementation was subsequently corrected so that invalid or unreadable images produce an appropriate error rather than being passed directly to the resizing operation.

6.13 Web Application Testing

The completed application was tested using both REAL and FAKE image samples.

The application successfully:

- Accepted image uploads.
- Processed valid images.
- Loaded the trained CNN model.
- Generated REAL/FAKE predictions.
- Generated Grad-CAM visualizations.
- Displayed the results through the Flask interface.

The system was also tested with an invalid image to verify error handling.

6.14 Implementation Summary

The implementation combines the complete machine learning workflow with a web-based interface.

The overall implementation can be summarized as:

Dataset



Image Preprocessing



CNN Training



Saved Model Weights



Flask Application



Image Upload



Preprocessing



CNN Prediction



Grad-CAM



Result Display

The completed implementation therefore provides an end-to-end prototype for AI-generated image classification and explainable prediction.

CHAPTER 7 — RESULTS AND DISCUSSION

7.1 Introduction

The performance of the implemented CIFAKE system was evaluated using a separate test dataset. The evaluation was performed using standard classification metrics including accuracy, precision, recall and F1-score. A confusion matrix was also generated to analyze the classification behaviour of the model.

In addition, the training and validation accuracy and loss were analyzed to understand the learning behaviour of the CNN model. The Flask application was tested using both REAL and FAKE images, and Grad-CAM visualizations were generated as part of the prediction process.

7.2 Dataset Evaluation

The processed dataset contains 100,000 images with two classes:

Class	Number of Images
-------	------------------

REAL	50,000
------	--------

FAKE	50,000
------	--------

Total	100,000
-------	---------

An 80:20 split was used for model evaluation.

Dataset	Images
---------	--------

Training	80,000
----------	--------

Testing	20,000
---------	--------

The balanced class distribution provides an equal number of REAL and FAKE images in the processed dataset.

7.3 Model Performance

The final model was evaluated on the 20,000-image test set.

The obtained results were:

Evaluation Metric Score	
Accuracy	95.93%
Precision	94.70%
Recall	97.29%
F1-Score	95.98%

The accuracy of 95.93% indicates that the model correctly classified the majority of images in the test dataset.

The precision of 94.70% indicates that a high proportion of the images predicted as the positive class were correctly classified.

The recall of 97.29% indicates that the model successfully identified a high proportion of the relevant positive samples.

The F1-score of 95.98% provides a combined measure of precision and recall and indicates strong overall classification performance.

7.4 Confusion Matrix

The confusion matrix obtained during evaluation was:

[[9456 544]
[271 9729]]

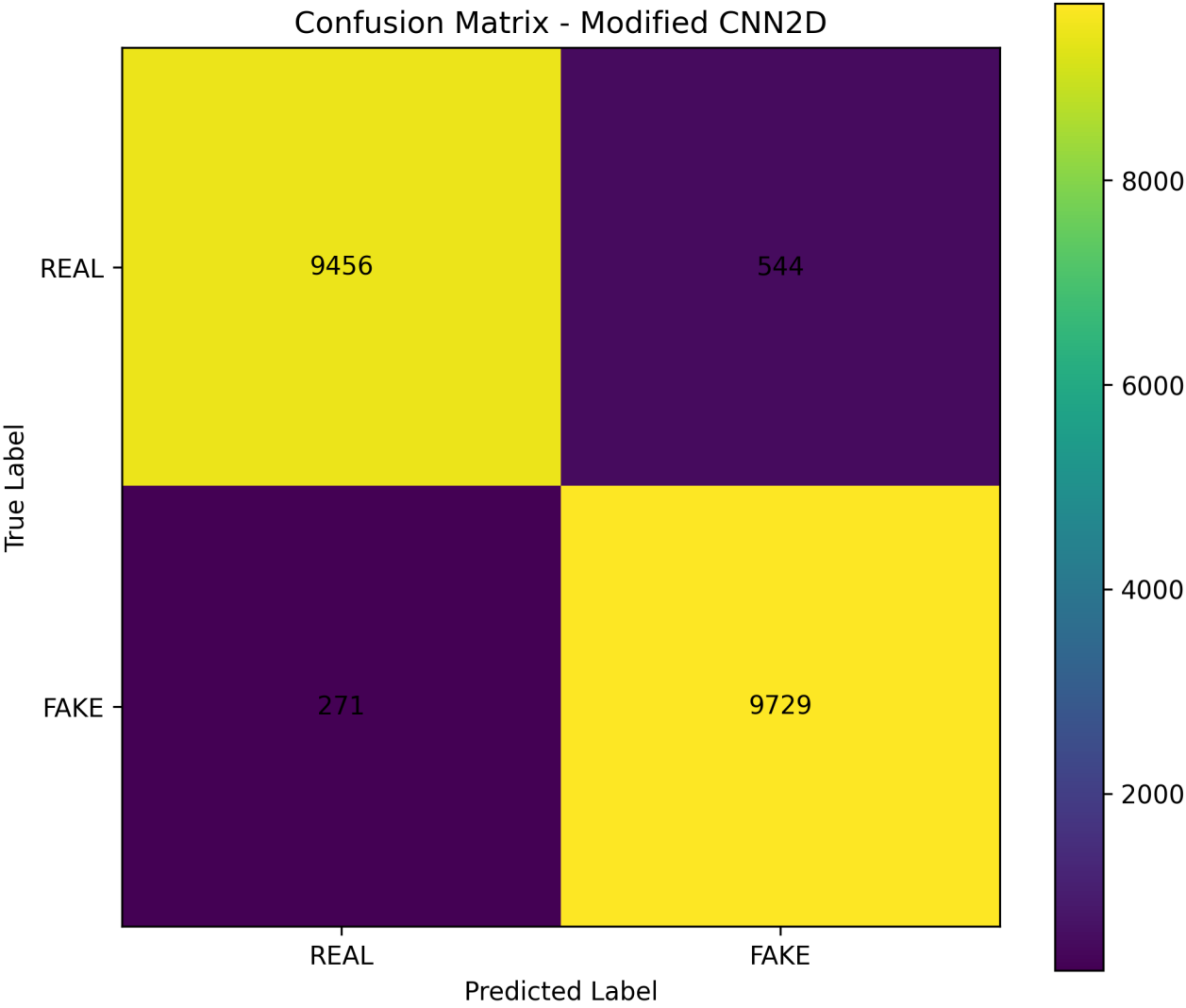


Figure 7.1: Confusion Matrix

The confusion matrix shows that 9,456 samples from one class and 9,729 samples from the other class were correctly classified.

A total of 815 samples were incorrectly classified.

The confusion matrix demonstrates that the model performs well for both classes, although some images remain difficult to classify correctly.

7.5 Training and Validation Accuracy

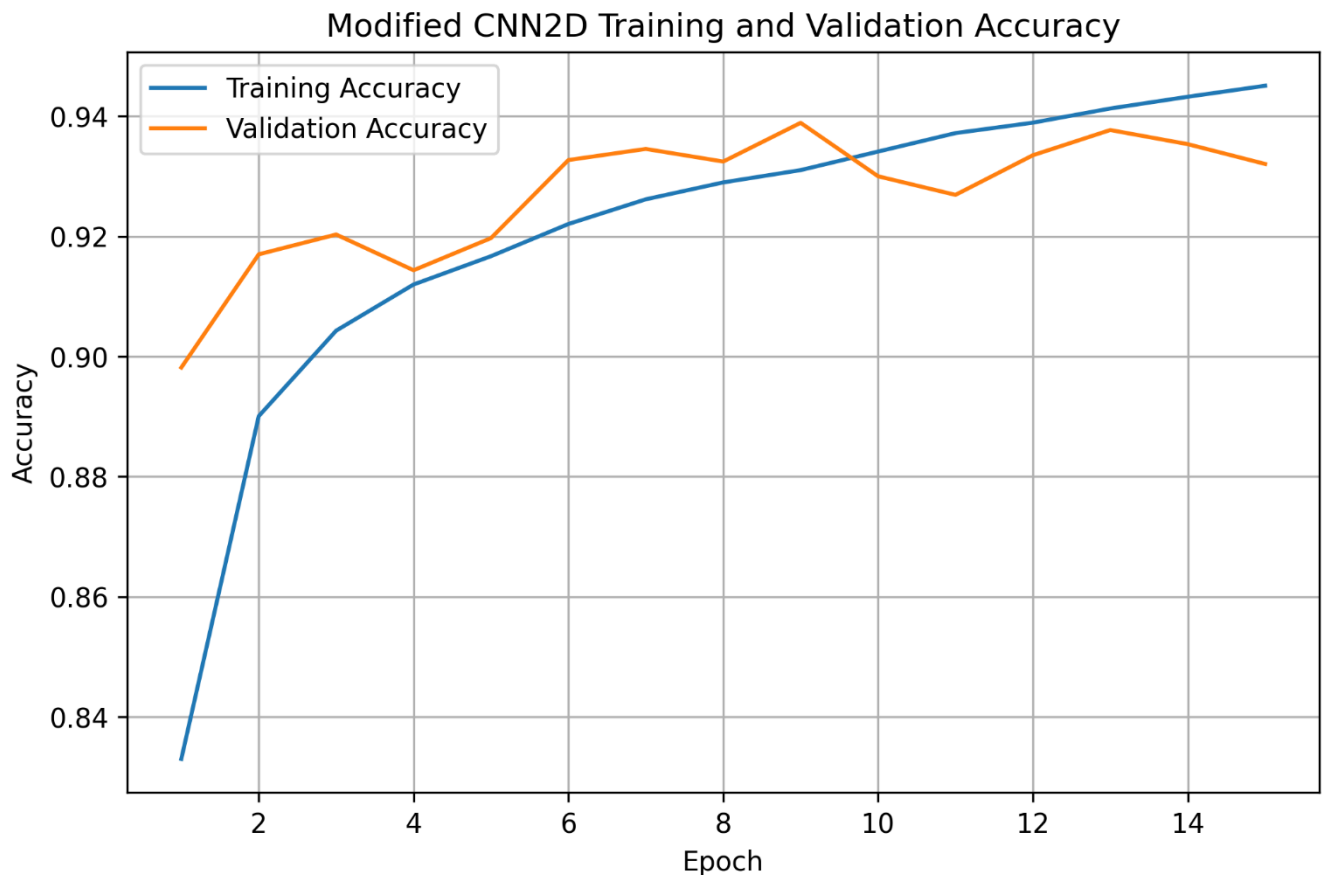


Figure 7.2: Training and Validation Accuracy

The training and validation accuracy graph shows how the model's classification performance changed during training.

The highest validation accuracy recorded in the saved training history was 93.89% at epoch 9.

At this epoch:

- Training accuracy = 93.10%
- Validation accuracy = 93.89%

The relatively close training and validation accuracy at this point indicates that the model was learning useful features without a large gap between training and validation performance.

7.6 Training and Validation Loss

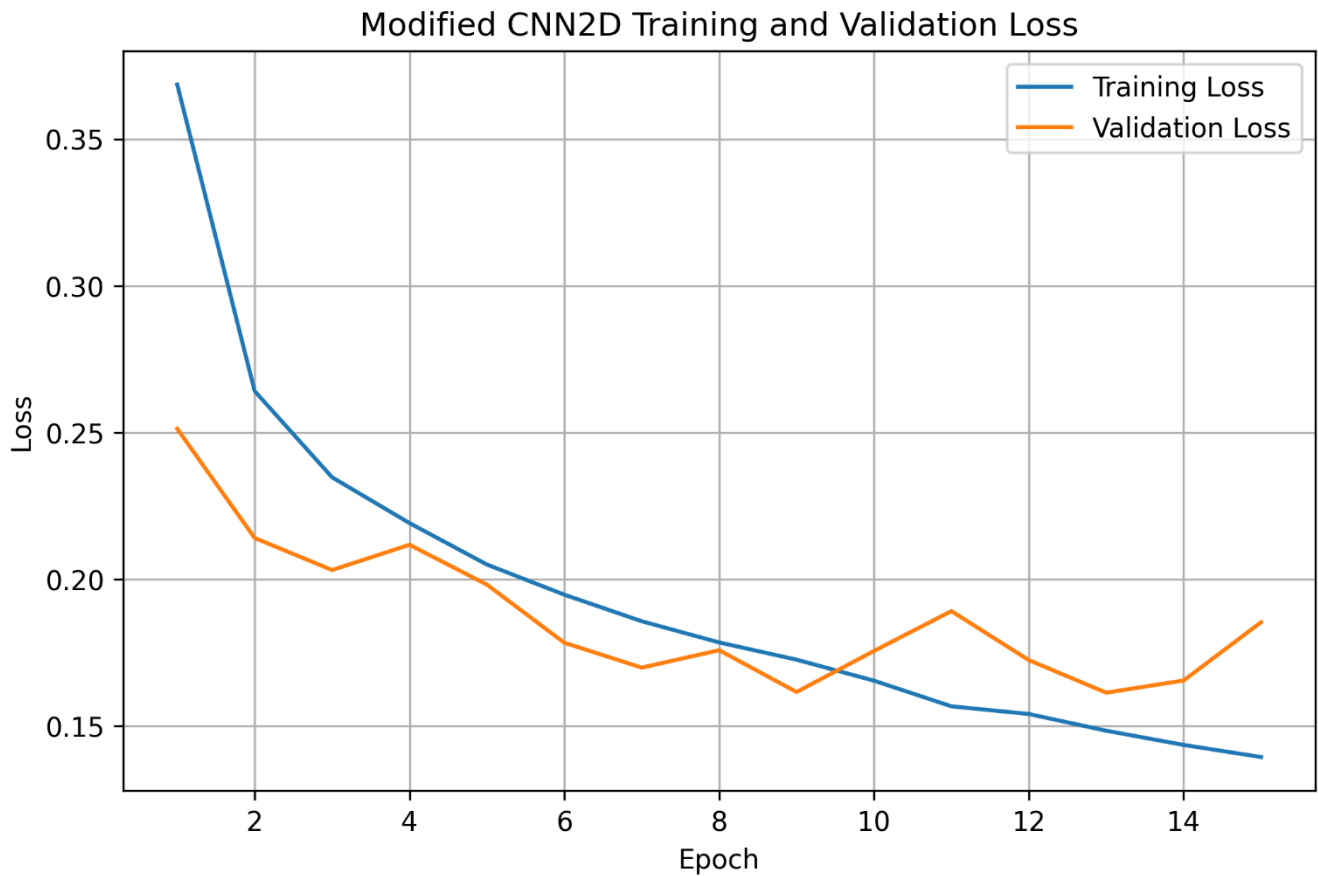


Figure 7.3: Training and Validation Loss

The loss graph represents the error of the model during training and validation.

At epoch 9, the recorded values were:

- Training loss = 0.1726
- Validation loss = 0.1615

The loss values provide an additional view of the model's learning behaviour and complement the accuracy graph.

7.7 REAL Image Prediction

The trained model was tested through the Flask web application using a REAL image.

The application successfully processed the uploaded image and produced:

Predicted As : REAL

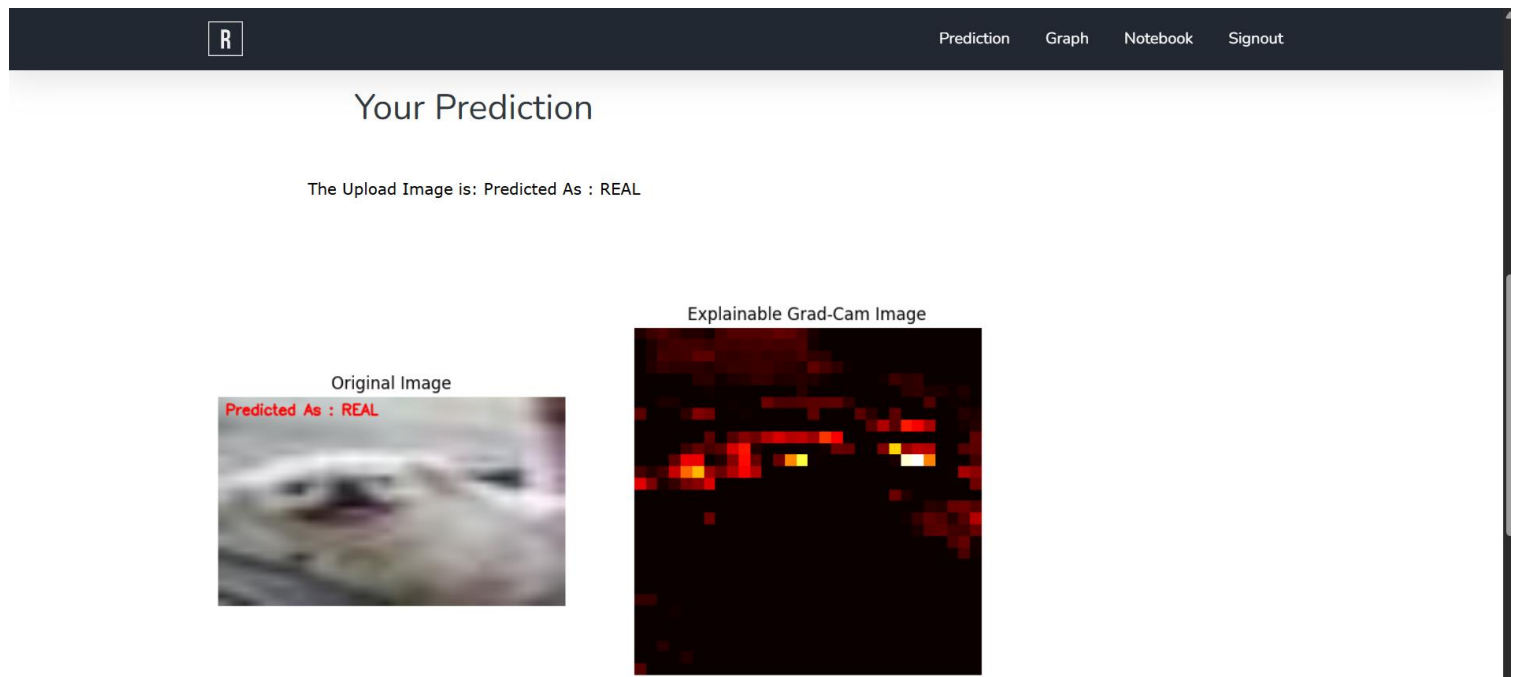


Figure 7.4: REAL Image Prediction

The result demonstrates that the trained model can be integrated with the web application and used for real-time image classification.

7.8 FAKE Image Prediction

The application was also tested using a FAKE image.

The system successfully processed the uploaded image and produced:

Predicted As : FAKE

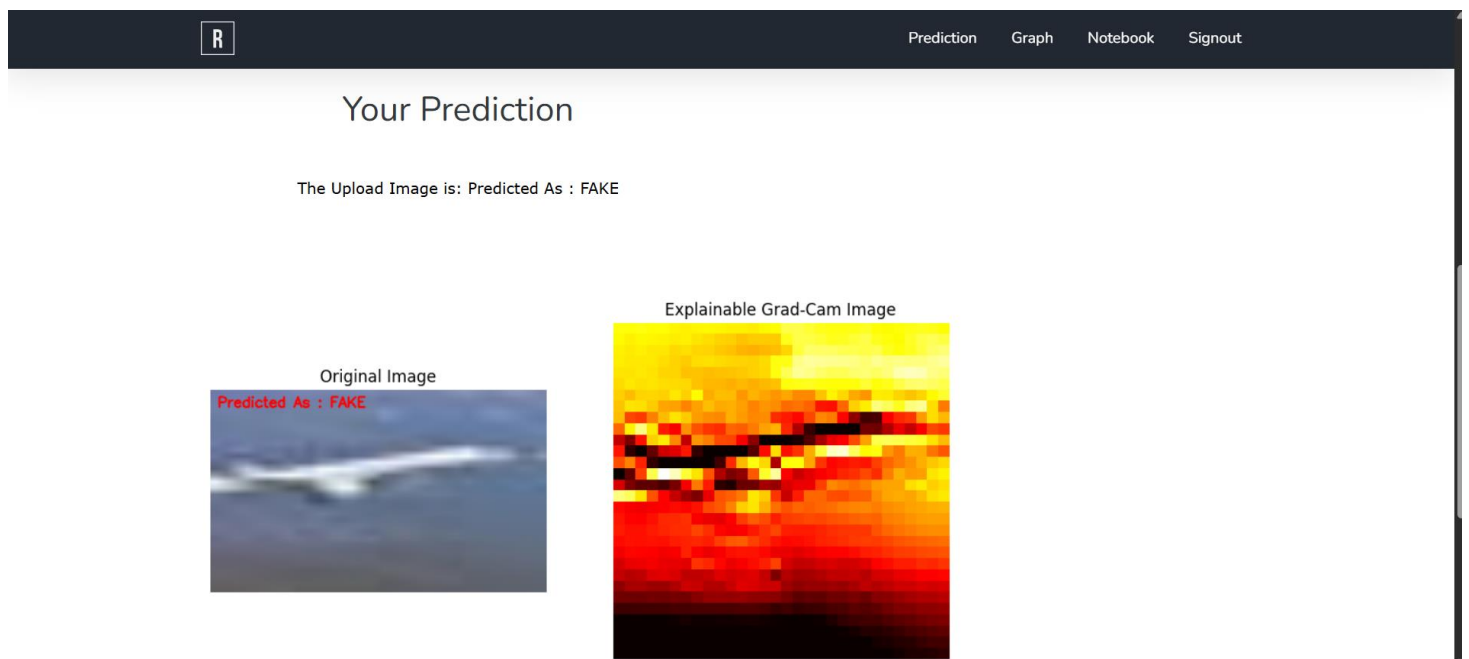


Figure 7.5: FAKE Image Prediction

This confirms that the application can identify both classes through the same prediction pipeline.

7.9 Grad-CAM Explainability

Grad-CAM was successfully integrated into the prediction process.

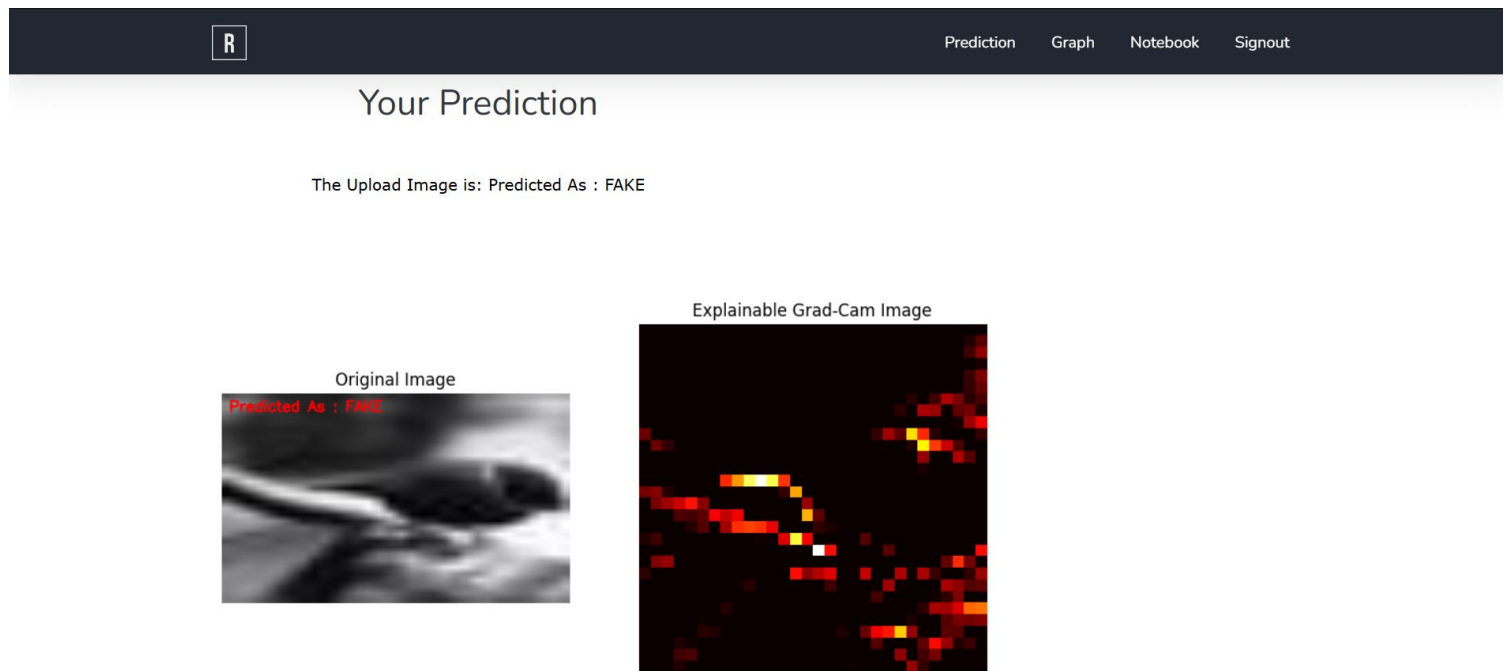


Figure 7.6: Grad-CAM Visualization

The Grad-CAM visualization provides a heatmap associated with the CNN's learned activation. It gives the user a visual representation of the regions that contributed to the model's response.

This improves the interpretability of the system because the application provides more information than a simple REAL or FAKE label.

7.10 Web Application Results

The Flask web application successfully performed the complete prediction workflow:

Image Upload



Image Validation



Image Preprocessing



CNN Prediction



REAL / FAKE Result



Grad-CAM Visualization

The application was successfully tested with both valid REAL and FAKE images.

An invalid image was also tested during development. Initially, an unreadable image caused OpenCV's resize operation to fail. The image-upload handling was subsequently improved so that invalid image input could be detected instead of being passed directly to the model.

7.11 Discussion

The evaluation results demonstrate that the implemented CNN model provides strong binary classification performance on the evaluated test set.

The 95.93% accuracy shows that the model correctly classified most of the 20,000 test images. The high recall of 97.29% indicates that the model was effective at identifying the relevant positive samples, while the precision of 94.70% indicates that most positive predictions were correct.

The F1-score of 95.98% provides a balanced representation of precision and recall.

However, the results should be interpreted within the scope of the dataset and evaluation procedure. Performance on new types of AI-generated images or images produced by unseen generative models may differ from the measured test performance.

7.12 Overall Result

The final evaluation confirms that the implemented CIFAKE system successfully combines:

- CNN-based image classification.
- Image preprocessing.
- Model evaluation.
- Flask-based deployment.
- REAL/FAKE prediction.
- Grad-CAM-based explainability.

The system therefore demonstrates a complete prototype for identifying AI-generated synthetic images through deep learning and providing an interpretable visualization of the model's prediction.

CHAPTER 8 — CHALLENGES AND LEARNING OUTCOMES

8.1 Challenges Faced

During the development and testing of the CIFAKE system, several technical challenges were encountered. These challenges involved the development environment, dataset organization, model integration, image processing and web application testing.

8.1.1 Dependency Compatibility

The original project used versions of libraries that were not completely compatible with the current Python environment. In particular, compatibility issues were encountered between Flask and Werkzeug.

The issue was resolved by installing compatible package versions and maintaining the project dependencies inside a Python virtual environment.

8.1.2 Dataset Organization

Initially, some dataset folders were not available in the main extracted project directory. The required dataset folders were located and organized correctly under the Dataset directory.

The final dataset structure was:

Dataset/

├── FAKE/

└── REAL/

The processed dataset was then verified to contain 100,000 images with balanced class labels.

8.1.3 Model and Processed Data Files

The trained model weights and processed dataset files needed to be available in the correct directories for the application and evaluation scripts.

The following important files were verified:

model/extension_weights.hdf5

model/extension_history.pkl

model/X.txt.npy

model/Y.txt.npy

Correct organization of these files allowed the model to be loaded without retraining every time the application was executed.

8.1.4 Image Upload Error

During application testing, an uploaded image initially caused an OpenCV error during the resizing operation because the image could not be decoded correctly.

The error occurred when OpenCV attempted to resize an empty image.

The image upload and validation logic was subsequently improved so that invalid or unreadable images are detected before the image is passed to the resizing and prediction stages.

8.1.5 Python Indentation Errors

During modification of the Flask prediction function, indentation errors were encountered in the Python source code.

These errors were identified using Python's compilation check:

```
python -m py_compile app.py
```

The indentation was corrected and the application was tested again.

8.1.6 CPU-Based Execution

TensorFlow reported that CUDA and NVIDIA GPU libraries were unavailable in the development environment.

As a result, the model was executed using the CPU. The system continued to function correctly, although CPU-based processing can be slower than GPU-based inference for larger workloads.

8.2 Solutions Implemented

The challenges were addressed using the following approaches:

- Created and used a dedicated Python virtual environment.
- Resolved package compatibility issues.
- Organized the dataset into REAL and FAKE directories.
- Verified the processed NumPy dataset.
- Verified the trained model weights and training history.
- Added image validation and improved upload handling.
- Used `py_compile` to identify Python syntax and indentation problems.
- Tested the Flask application repeatedly after modifications.
- Used CPU execution when GPU acceleration was unavailable.

8.3 Technical Skills Learned

The project provided practical experience in several areas of software development and artificial intelligence.

Machine Learning and Deep Learning

The project provided experience with:

- CNN architecture.
- Image classification.
- Model training.
- Model evaluation.
- Training and validation analysis.
- Classification metrics.

Computer Vision

Practical experience was gained in:

- Image loading.
- Image validation.
- Image resizing.
- Pixel normalization.

- OpenCV-based image processing.

Explainable AI

The project provided an understanding of:

- Explainable Artificial Intelligence.
- Grad-CAM.
- CNN activation visualization.
- Interpreting model predictions.

Web Development

The Flask implementation provided experience with:

- Flask routes.
- HTTP requests.
- File uploads.
- Template rendering.
- Connecting machine learning models to web applications.

Debugging and Problem Solving

The project also provided practical experience in identifying and resolving:

- Dependency errors.
- Python indentation errors.
- Missing project files.
- Image decoding errors.
- Model-loading issues.
- Runtime application errors.

8.4 Overall Learning Outcome

The project provided practical exposure to the complete workflow of a machine learning application.

The development process covered dataset preparation, image preprocessing, CNN-based classification, model evaluation, explainability, web integration and application testing.

The project also demonstrated that developing a practical AI system requires more than training a model. Proper data preparation, model evaluation, error handling, explainability and user-interface integration are important parts of a complete machine learning application.

The experience gained from this project provides a foundation for developing more advanced computer vision and explainable AI applications in the future.

CHAPTER 9 — FUTURE ENHANCEMENTS

9.1 Introduction

The current CIFAKE system provides a working prototype for classifying images as REAL or FAKE and generating a Grad-CAM-based visual explanation. Although the implemented system provides strong performance on the evaluated dataset, several improvements can be explored in future versions.

9.2 Advanced CNN Architectures

Future implementations can investigate more advanced deep learning architectures such as ResNet, EfficientNet, MobileNet and Vision Transformers.

These architectures can be compared with the current Modified CNN2D model to determine whether deeper or more efficient feature extraction can improve classification performance.

9.3 Higher-Resolution Image Analysis

The current implementation resizes input images to 32×32 pixels. Future versions could experiment with higher-resolution inputs.

Higher-resolution processing may preserve additional visual details and subtle artifacts that could be useful for identifying synthetic images.

9.4 Improved Generalization

The current model is evaluated using the processed CIFAKE dataset. Future work can evaluate the system using images generated by AI models that were not represented in the training dataset.

Testing on unseen generative models would provide a better understanding of how well the detector generalizes to newly developed image-generation techniques.

9.5 Multiple Synthetic Image Categories

The current system performs binary classification:

REAL

or

FAKE

A future version could classify synthetic images into multiple categories based on their possible generation source or model family.

For example, the system could potentially distinguish between images generated using different types of generative AI systems.

9.6 Advanced Explainability Techniques

Grad-CAM is currently used to provide visual explanations.

Future versions could investigate additional explainability techniques such as:

- Grad-CAM++.
- Integrated Gradients.
- SHAP-based explanations.
- Comparative visualization of multiple explanation methods.

Comparing different XAI techniques could provide a more detailed understanding of the features influencing the model's decisions.

9.7 Confidence Score

The current application displays the predicted class. A future version could also display the model's prediction confidence.

For example:

Prediction : FAKE

Confidence : 96.4%

Providing confidence information could help users understand how strongly the model supports a particular prediction.

9.8 Batch Image Classification

The current system processes an uploaded image individually.

Future development could add batch processing, allowing users to upload multiple images and receive predictions for all of them in a single operation.

This would make the system more practical for applications involving large numbers of images.

9.9 Cloud Deployment

The Flask application could be containerized using Docker and deployed on cloud infrastructure.

A cloud-based version could allow users to access the system remotely and could be scaled to support a larger number of simultaneous requests.

The machine learning model could also be separated into a dedicated inference service for improved scalability.

9.10 API Integration

A REST API could be added to allow external applications to use the image classification service programmatically.

Such an API could accept an image as input and return information such as:

Predicted Class

Confidence Score

Explainability Information

This would make the CIFAKE detector easier to integrate into other software systems.

9.11 Security and Reliability Improvements

Future versions can also improve the security and reliability of the web application.

Possible improvements include:

- Secure password storage using hashing.
- Improved authentication and authorization.
- File-size restrictions.
- Stronger file-type validation.
- Secure handling of uploaded files.
- Protection against malicious file uploads.

These improvements would make the application more suitable for deployment in a real-world environment.

9.12 Research Opportunities

Further research can investigate the robustness of AI-generated image detection against common image transformations such as:

- Compression.
- Resizing.
- Cropping.
- Image enhancement.
- Noise addition.

Research can also investigate whether combining multiple models or using ensemble approaches can improve the reliability of synthetic image detection.

9.13 Future Scope Summary

The future development of the CIFAKE system can focus on improving accuracy, generalization, explainability, scalability and security.

The current project provides a foundation that can be extended toward a more advanced AI-generated image detection platform capable of handling newer generative models, higher-resolution images and larger-scale image analysis.

CHAPTER 10 — CONCLUSION

10.1 Conclusion

The CIFAKE project successfully demonstrates the use of deep learning and explainable artificial intelligence for identifying AI-generated synthetic images.

A Modified CNN2D model was implemented to classify images into two categories: REAL and FAKE. The images were preprocessed by resizing them to 32×32 pixels and normalizing their pixel values before being provided to the model.

The trained model was evaluated using a reproducible test set containing 20,000 images. The system achieved an accuracy of 95.93%, with a precision of 94.70%, recall of 97.29% and F1-score of 95.98%.

The project also integrated Grad-CAM to provide a visual explanation of the CNN's prediction. Instead of displaying only a classification label, the system generates a heatmap that helps indicate the regions associated with the model's learned response.

A Flask-based web application was developed to make the system accessible through a browser. Users can upload an image and receive a REAL or FAKE prediction together with the explainability visualization.

10.2 Major Achievements

The major achievements of the project are:

1. Successfully implemented a CNN-based REAL/FAKE image classification system.
2. Processed and evaluated a balanced dataset containing 100,000 images.
3. Implemented image preprocessing using OpenCV and NumPy.
4. Integrated the trained CNN model into a Flask web application.
5. Successfully tested both REAL and FAKE image predictions.
6. Integrated Grad-CAM for explainable predictions.
7. Evaluated the model using accuracy, precision, recall and F1-score.
8. Generated a confusion matrix and training/validation graphs.
9. Implemented basic image validation and error handling.
10. Created an end-to-end prototype combining deep learning, computer vision, explainable AI and web development.

10.3 Final Outcome

The completed system demonstrates that a CNN can be used effectively for binary classification of real and AI-generated images within the evaluated dataset.

The combination of CNN classification and Grad-CAM explainability makes the system more informative than a conventional classifier that provides only a prediction label.

The project also provided practical experience with the complete development lifecycle of an AI application, including dataset preparation, preprocessing, model evaluation, explainability, web integration, debugging and application testing.

Although the current system is a prototype and its performance may vary when applied to unseen generative models or different image distributions, it provides a strong foundation for future development in AI-generated image detection.

REFERENCES

1. Bird, J. J., & Lotfi, A. "CIFAKE: Image Classification and Explainable Identification of AI-Generated Synthetic Images." arXiv, 2023.
2. Selvaraju, R. R., Cogswell, M., Das, A., Vedantam, R., Parikh, D., & Batra, D. "Grad-CAM: Visual Explanations from Deep Networks via Gradient-Based Localization." Proceedings of the IEEE International Conference on Computer Vision (ICCV), 2017.
3. TensorFlow Documentation. TensorFlow: An End-to-End Open Source Machine Learning Platform.
4. Keras Documentation. Keras Deep Learning API Documentation.
5. Flask Documentation. Flask Web Development Framework Documentation.
6. OpenCV Documentation. OpenCV – Open Source Computer Vision Library.